© Guven

Audio Generation with Hugging Face Models

Text-to-Speech Synthesis with Suno/Bark

This code demonstrates a text-to-speech (TTS) synthesis using the `transformers` library from Hugging Face, specifically leveraging the "Bark" model from Suno. The goal of this code is to convert a text prompt into spoken audio that can be played directly in a Jupyter notebook.

A TTS pipeline is created using the Hugging Face `pipeline` function, specifying "text-to-speech" as the task and "suno/bark" as the model. The pipeline will run on the selected device (either GPU or CPU). This setup abstracts many complexities and simplifies running inference with the model.

The TTS pipeline parameter `do_sample=True` is specified to enable sampling, which adds slight variability to the output and makes it sound more natural.

The sampling rate of the generated audio is accessed from the model's configuration. The audio is then converted to a compatible format (if necessary) and played directly within the notebook using IPython's `Audio` display feature. This step allows for immediate playback and easy experimentation.

🔶 Model Overview

The `suno/bark` model is a text-to-speech model available on the Hugging Face Hub. It is trained to generate realistic speech from text inputs, leveraging advanced deep learning methods and a vast dataset. This model is designed for high-quality, natural-sounding audio generation, making it suitable for a wide range of speech synthesis applications.

🔶 Inference

The model takes in a text input, processes it, and generates an audio waveform as the output. This audio waveform is compatible with playback and can also be saved for later use.

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
```

```
plt.rcParams['figure.dpi'] = 72
import numpy as np
import sys
import os
import warnings

MODEL_PATH= '/EP_models/'
os.environ['HF_HOME']= MODEL_PATH  # before import transformers
os.environ['HF_DATASETS_CACHE']= MODEL_PATH  # dataset cache
os.environ['HF_HUB_DISABLE_SYMLINKS_WARNING'] = "1"  # symlinks are better
```

In [2]:
```
import torch
import transformers
from transformers import pipeline, AutoProcessor, AutoModel
from transformers import logging as hf_logging

import scipy.io.wavfile

from IPython.display import Audio

Device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

# filter warnings
hf_logging.set_verbosity_error()

warnings.filterwarnings("ignore", message="To copy construct from a tensor

print(f'transformers version= {transformers.__version__}')
print(f'PyTorch version= {torch.__version__}')
print(f'CUDA available= {torch.cuda.is_available()}')
```

```
transformers version= 4.57.6
PyTorch version= 2.9.1+cu130
CUDA available= True
```

In [3]:
```
# loads the model
Synthesiser = pipeline('text-to-speech', 'suno/bark', device=Device)
```

In [4]:
```
# generate speech
def gen_speech(_text):
    speech = Synthesiser(_text, forward_params={"do_sample": True})
    sampling_rate = Synthesiser.model.generation_config.sample_rate  # acc
    return speech, sampling_rate
```

In [5]:
```
# run a few times

speech, rate = gen_speech("Hello, my course is cooler than yours!")

# play the audio directly in the notebook
# if speech['audio'] is a PyTorch tensor and on the GPU, move it to CPU wi
Audio(speech['audio'].squeeze(), rate=rate)
```

Out[5]:

0:00 / 0:04

In [6]:
```
# save the wav file
scipy.io.wavfile.write('output1.wav', rate, speech['audio'].squeeze())
```

Text-to-Speech Synthesis Using AutoProcessor and AutoModel

In this example, the code achieves text-to-speech synthesis using the `AutoProcessor` and `AutoModel` classes from Hugging Face. These classes provide more granular control over model and input handling than the simplified `pipeline` approach.

The model and processor are loaded directly from the "suno/bark" repository, with the model explicitly moved to the chosen device (GPU or CPU). Text input is processed by the processor, which converts it to the required tensor format and moves it to the same device as the model, ensuring compatibility during processing.

To generate speech, the `.generate` method of the model is used within a `torch.no_grad()` block, which turns off gradient calculation and optimizes memory usage during inference. This approach allows for greater flexibility in adjusting inference parameters or handling the output format directly.

The generated speech tensor is transferred back to the CPU (if processed on the GPU) to enable compatibility with audio playback. Unlike the `pipeline` function, this method requires manually converting the output tensor to a format compatible with `IPython.display.Audio`. Here, the sampling rate is set to 24 kHz, the standard for the Bark model, to ensure accurate playback.

In [7]:
```
# input processor and the model
Processor = AutoProcessor.from_pretrained('suno/bark')
Model = AutoModel.from_pretrained('suno/bark').to(Device)  # move the mode
```

In [8]:
```
# run a few times

# prepare inputs and move them to the same device as the model
inputs = Processor(
    text=["Hello, my name is Suno. And, uh — and I like pizza. [laughs] Bu
    return_tensors="pt",
).to(Device)  # move inputs to GPU

# generate speech on the specified device
```

```python
with torch.no_grad():
    speech_values = Model.generate(**inputs, do_sample=True)

# move generated speech back to CPU for audio processing (if necessary)
speech_values = speech_values.cpu()

# you may need to convert the output to the format expected by your audio
# (e.g., numpy array, bytes, etc.)
# check the documentation for your audio player for specific requirements

# assume similar format to pipeline output for audio handling
audio_data = speech_values.numpy().squeeze()

# The sampling rate for Bark is fixed at 24kHz
sampling_rate = 24000  # Bark's default sampling rate

# play
Audio(audio_data, rate=sampling_rate)
```

Out[8]:

0:00 / 0:13

In [9]:

```python
# save the wav file
scipy.io.wavfile.write("output2.wav", sampling_rate, audio_data)
```

In [10]:

```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```